Homework 1 NYT Text Classification

Aozhe Zhang | Computer Science and Technology
Student ID: 2313447

Abstract. This report presents the implementation and results of the three tasks in Homework 1, covering methods for classifying 11,519 New York Times articles into business, politics, and sports.The methods include binary Bag-of-Words and word frequency with Logistic Regression, averaged 100-dimensional GloVe and Word2Vec embeddings with Logistic Regression, and BERT fine-tuned for three epochs with a maximum sequence length of 64 tokens. Binary Bag-of-Words achieves the highest test Macro-F1 of 97.28% with 98.87% Accuracy. Word2Vec trained on NYT leads the embedding methods at 97.18% Macro-F1, while BERT reaches 96.05%, with errors concentrated between business and politics. The results highlight the effectiveness of lexical features and in-domain embeddings, and the importance of input and training settings.

Keywords: text classification; Bag-of-Words; GloVe; Word2Vec; BERT

1 Introduction

News classification assigns a category to an article according to its content. It provides a concrete setting for studying a central question in natural language processing: how should variable-length text be represented so that a model can learn useful distinctions? Articles are sequences of discrete symbols, whereas most classifiers operate on numerical features. The choice of representation determines which information is retained and which information is discarded before a prediction is made.

This assignment studies three representation families. Sparse lexical features encode the presence or frequency of individual words. They preserve explicit vocabulary evidence and offer a simple baseline. Static embeddings map words to dense vectors whose geometry reflects corpus-level usage; a document can then be represented by averaging its word vectors. Contextual models instead compute token representations from the input sequence, allowing the representation of a word to depend on its surrounding text. BERT also transfers information learned during large-scale pre-training [1].

Formally, the supervised dataset consists of pairs (d_i, y_i), where d_i is an article and y_i belongs to one of three classes. A model predicts

$$\hat{y}_i = \arg\max_{k\in\{1,2,3\}} p_\theta(y = k \mid d_i). \tag{1}$$

For Tasks 1 and 2, a fixed representation $\phi(d_i)$ is passed to Logistic Regression (LR). Task 3 learns a BERT encoder and classification head jointly.

2 Experimental Setup

2.1 Datasets and shared partition

The supplied NYT dataset contains 11,519 articles labelled business, politics, or sports. AG News supplies 90,000 text rows for unsupervised Word2Vec training in Task 2.

Two shuffled, stratified splits with `random_state=42` allocate 80% of NYT rows to training, then divide the remaining 20% equally between validation and test. All six methods use the same row indices. Stratification preserves the class proportions, but it does not remove the dataset's imbalance: sports represents approximately three quarters of the articles. Table 1 gives the class counts.

Table 1. NYT samples in the shared split.

Class	Train	Validation	Test
Business	1,143	143	143
Politics	1,161	145	145
Sports	6,911	864	864
Total	9,215	1,152	1,152

2.2 Preprocessing and selection protocol

Tasks 1 and 2 share the same preprocessing function: `word_tokenize` from NLTK. It lowercases the input, handles English contractions, and retains punctuation tokens because no additional filtering is applied. There is no stop-word removal, stemming, or lemmatization. The notebook checks the English tokenizer resources and downloads `punkt` and `punkt_tab` if necessary. BERT uses its own uncased WordPiece tokenizer, independently of this function. The report describes the revised implementation; these preprocessing choices are not additional assignment requirements.

The BoW vocabulary and NYT Word2Vec model are fitted only on training texts. For every LR experiment, validation Macro-F1 selects $C \in \{0.1, 1, 10\}$, after which the selected classifier is fitted on training data only. BERT's checkpoint is selected by the same validation metric. The test set is reserved for evaluating the selected models and does not participate in hyperparameter or checkpoint selection.The constants are `SEED=42`, `WORD_VECTOR_SIZE=100`, `MAX_LENGTH=64`, and `BERT_EPOCHS=3`.

2.3 Evaluation metrics

Accuracy measures the proportion of correct predictions:

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}[\hat{y}_i = y_i]. \tag{2}$$

Macro-F1 gives equal weight to each of the three class F1 scores:

$$\text{Macro-F1} = \frac{1}{3}\sum_{k=1}^{3}\frac{2TP_k}{2TP_k + FP_k + FN_k}. \tag{3}$$

Here TP_k, FP_k, and FN_k are the true-positive, false-positive, and false-negative counts for class k. Both Accuracy and Macro-F1 are reported to account for class imbalance and provide a more comprehensive evaluation of classification performance.

3 Task 1: Bag-of-Words

3.1 Principles of lexical representations

Bag-of-Words (BoW) represents each document as a fixed-dimensional vector based on a vocabulary, ignoring word order. This experiment considers two representations:

$$x_j^{\text{binary}}(d) = \mathbb{I}[c_j(d) > 0], \qquad x_j^{\text{frequency}}(d) = c_j(d),$$

where $c_j(d)$ denotes the occurrence count of word j in document d. Binary BoW records word presence, whereas Word Frequency preserves occurrence counts, providing additional information about word importance while being sensitive to document length.

3.2 Logistic Regression classifier

For multiclass text classification, Logistic Regression (LR) computes class probabilities using the softmax function:

$$p(y = k \mid \mathbf{x}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x} + b_k)}{\sum_{\ell=1}^{3}\exp(\mathbf{w}_\ell^\top \mathbf{x} + b_\ell)}.$$

The model is trained with regularization controlled by C, where smaller values indicate stronger regularization. Both Tasks 1 and 2 use LR with the same hyperparameter selection procedure to ensure a consistent comparison across text representations.

3.3 Implementation

The NLTK tokenizer introduced in Section 2 is shared by both BoW representations. A `CountVectorizer` fitted only on NYT training articles produces a vocabulary of 127,053 tokens and a sparse count matrix of size $9{,}215 \times 127{,}053$. Validation and test articles are transformed using the same vocabulary.

As shown in Listing 1, the count matrices provide the Word Frequency features. Binary BoW features are obtained by copying these matrices and setting all nonzero entries to one, preserving the original counts for the second experiment.

Listing 1. Task 1 feature construction.

```
def tokenize(text):
    return word_tokenize(
        text.lower(), language="english"
    )

vectorizer = CountVectorizer(
    tokenizer=tokenize,
    token_pattern=None,
    lowercase=False,
    dtype=np.int32,
)
X_train_count = vectorizer.fit_transform(train_text)
X_val_count = vectorizer.transform(val_text)
X_test_count = vectorizer.transform(test_text)

X_train_binary = X_train_count.copy()
X_val_binary = X_val_count.copy()
X_test_binary = X_test_count.copy()
for matrix in (X_train_binary, X_val_binary,
               X_test_binary):
    matrix.data[:] = 1
```

Both representations use the LR selection procedure in Listing 2. Models with $C \in \{0.1, 1, 10\}$ are fitted on training data and compared using validation Macro-F1. The selected model is then evaluated on the test set. Task 2 reuses this procedure with dense document vectors.

Listing 2. Validation-based LR selection.

```
def fit_logistic_regression(
    X_train, y_train, X_val, y_val
):
    candidates = [0.1, 1.0, 10.0]
    scores = []
    for c_value in candidates:
        model = LogisticRegression(
            C=c_value, max_iter=1000,
            solver="lbfgs", random_state=SEED,
        )
        model.fit(X_train, y_train)
        score = f1_score(
            y_val, model.predict(X_val),
            average="macro", zero_division=0,
        )
        scores.append(score)
    best_c = candidates[int(np.argmax(scores))]
    final_model = LogisticRegression(
        C=best_c, max_iter=1000,
        solver="lbfgs", random_state=SEED,
    )
    final_model.fit(X_train, y_train)
    return final_model, best_c
```

3.4 Results and analysis

Table 2 shows the selected regularization parameters and test scores. Binary BoW achieves 98.87% Accuracy and 97.28% Macro-F1, exceeding Word Frequency by 0.12 percentage points in Macro-F1 and ranking highest among all six methods. Both representations perform strongly, indicating that lexical features provide effective signals for news classification. Binary BoW's slightly better performance suggests that word presence captures the key topic information, while retaining occurrence counts provides little additional benefit.

Table 2. Task 1 NYT test results.

Representation	C	Acc. (%)	Macro-F1 (%)
Binary BoW	1	**98.87**	**97.28**
Word Frequency	0.1	98.78	97.16

4 Task 2: Word2Vec

4.1 From word embeddings to document representations

Word embeddings replace vocabulary-sized indicator vectors with dense, low-dimensional vectors. Their central motivation is that words used in related contexts can have related representations. Unlike BoW, a dense representation can encode similarities between different words, but the classifier still requires a fixed-size representation for each article.

This task uses the mean of the available 100-dimensional word vectors:

$$\mathbf{e}(d) = \frac{1}{|T_d|} \sum_{w \in T_d} \mathbf{e}(w), \tag{4}$$

where T_d is the multiset of tokens with an embedding. Repeated words contribute repeatedly, and out-of-vocabulary (OOV) tokens are skipped. A document with no valid tokens receives a zero vector. Mean pooling is inexpensive and handles variable document length, but it removes word order and compresses all retained lexical evidence into 100 coordinates. LR then classifies the pooled vector.

4.2 GloVe: global co-occurrence information

GloVe learns static word vectors from corpus-wide word co-occurrence statistics [3]. Its objective relates vector dot products and bias terms to the logarithm of co-occurrence counts, with a weighting function that controls the influence of different counts. The resulting vectors capture information aggregated across a corpus rather than deriving a new representation for each occurrence of a word.

The implementation uses the required pre-trained `glove.6B.100d.txt`. The loader retrieves vectors for 66,431 of the 144,968 distinct NLTK tokens observed across NYT. Looking up held-out words in this fixed resource does not train or update the vectors and does not use held-out labels. Document means and LR are then computed using the shared protocol.

4.3 Word2Vec: local context prediction

Word2Vec learns word vectors through predictive objectives. In the skip-gram formulation, a center word is used to predict nearby context words [2]. For a token sequence $(w_1, \ldots, w_L)$, the conceptual objective is to increase

$$\sum_{t=1}^{L} \sum_{\substack{-r \le j \le r \\ j \neq 0}} \log p(w_{t+j} \mid w_t), \tag{5}$$

with valid context positions only. This describes the skip-gram principle; the notebook delegates training and its computational approximation to Gensim. The context window supplies local distributional evidence, and the learned vectors depend on the corpus used for training.

Two models use the same explicit settings in Table 3. The first is trained on all 90,000 AG News texts, without labels, and produces a 43,706-token vocabulary. The second is trained only on the 9,215 NYT training articles, producing 57,117 tokens. NYT validation and test articles do not participate in either NYT Word2Vec training or LR fitting. Options not explicitly set in the notebook use the installed Gensim defaults.

Table 3. Explicit Word2Vec settings for both corpora.

Parameter	Value
Architecture	Skip-gram (`sg=1`)
Vector dimensions	100
Context window	5
Minimum word count	2
Training epochs	5
Workers / random seed	1 / 42

4.4 Core implementation

Listing 3 shows the shared pooling routine. `present` contains one vector for each matched token occurrence, so repeated words retain their multiplicity. The membership test skips OOV tokens, and `np.mean(..., axis=0)` averages along the token axis to obtain one 100-dimensional vector. Rows are initialized to zero; an article with no matched token therefore keeps its zero row. Coverage and empty-document counters are omitted here, but are computed in the original routine.

Listing 3. Document mean pooling.

```python
def mean_document_vectors(
    texts, keyed_vectors,
    dimension=WORD_VECTOR_SIZE,
):
    vectors = np.zeros(
        (len(texts), dimension), dtype=np.float32
    )
    for row, text in enumerate(texts):
        tokens = tokenize(text)
        present = [
            keyed_vectors[word] for word in tokens
            if word in keyed_vectors
        ]
        if present:
            vectors[row] = np.mean(present, axis=0)
    return vectors
```

Listing 4 connects the three resources to the shared experiment runner. `load_needed_glove` scans the fixed embedding file and keeps only required words, checking that each retained vector has 100 components. `AGTokenCorpus` reopens the AG CSV on each iteration and yields NLTK tokens from nonblank text rows. `NYTTrainTokenCorpus` instead iterates over `train_text` only. These repeatable iterators supply Word2Vec without keeping the full tokenized corpus resident in memory.

Listing 4. Three embedding pipelines.

```python
glove_vectors = load_needed_glove(
    ensure_glove_100d(),
    train_text + val_text + test_text,
)
glove_model, glove_c, glove_pred = \
    run_embedding_experiment(
        "GloVe 100d mean + LR", glove_vectors
    )

ag_word2vec = Word2Vec(
    sentences=AGTokenCorpus(),
    vector_size=WORD_VECTOR_SIZE, window=5,
    min_count=2, workers=1, sg=1,
    epochs=5, seed=SEED,
)
ag_model, ag_c, ag_pred = run_embedding_experiment(
    "AG Word2Vec 100d mean + LR", ag_word2vec.wv
)

nyt_word2vec = Word2Vec(
    sentences=NYTTrainTokenCorpus(),
    vector_size=WORD_VECTOR_SIZE, window=5,
    min_count=2, workers=1, sg=1,
    epochs=5, seed=SEED,
)
nyt_w2v_model, nyt_w2v_c, nyt_w2v_pred = \
    run_embedding_experiment(
        "NYT Word2Vec 100d mean + LR",
        nyt_word2vec.wv,
    )
```

The `.wv` attribute exposes a trained Word2Vec model's keyed vectors; GloVe supplies a dictionary with the same lookup operations used by pooling. `run_embedding_experiment`, defined in the Task 2 helper cell, computes document means for each split, calls the LR helper, predicts test labels, and records Accuracy and Macro-F1. Thus, the three pipelines share the downstream classifier and differ in their embedding source. The constructor's `sg=1` explicitly selects skip-gram.

4.5 Results and analysis

All three embedding pipelines select $C = 10$. Table 4 reports performance alongside token-occurrence coverage, defined as the fraction of test tokens for which a vector is available. Coverage describes how much text survives OOV filtering.

Table 4. Task 2 NYT test results and token coverage.

Embedding source	Cov. (%)	Acc. (%)	Macro-F1 (%)
GloVe	98.05	98.52	96.37
AG News Word2Vec	95.48	97.92	94.74
NYT Word2Vec	98.36	**98.87**	**97.18**

NYT Word2Vec has the highest Macro-F1 among the embedding sources, exceeding AG News Word2Vec by 2.44 points and GloVe by 0.81 points. Its advantage over AG News coincides with higher coverage and an in-domain training corpus. NYT Word2Vec also has the highest test coverage in this revised run. Coverage and classification quality remain different properties: the former counts matched token occurrences, including punctuation retained by NLTK, and does not evaluate the information encoded in each vector.

NYT Word2Vec is only 0.10 Macro-F1 points below Binary BoW and 0.02 points above Word Frequency. This near tie shows that in-domain mean embeddings can be competitive under the shared protocol. GloVe and AG Word2Vec perform less well.

5 Task 3: Pre-trained Neural Models

5.1 Contextual encoding and transfer learning

BERT is a bidirectional Transformer encoder pre-trained with masked language modeling and next-sentence prediction [1]. Self-attention allows a token representation to incorporate information from other positions in the input. Its contextual representations therefore differ from the fixed word vectors in Task 2. Fine-tuning adapts this pre-trained encoder to the labelled classification task rather than learning all language representations from scratch.

The implementation initializes BERT with a new three-class classification head. The pooled representation associated with the special `[CLS]` token is passed to that head; softmax scores are trained with cross-entropy against the article labels. All encoder and head parameters are updated. This distinguishes full fine-tuning from freezing BERT and fitting a separate LR classifier.

5.2 Input and optimization settings

The required maximum sequence length is **64**, and training runs for **three complete epochs**. Uncased WordPiece tokenization applies truncation and fixed padding. For a single article, the 64 positions include `[CLS]` and `[SEP]`, leaving at most 62 content tokens. Table 5 lists the explicit optimization settings.

Table 5. BERT input and training configuration.

Setting	Value
Maximum sequence length	64
Epochs	3
Optimizer	AdamW
Learning rate	2×10^{-5}
Weight decay	0.01
Train / evaluation batch size	16 / 64
Maximum gradient norm	1.0
Checkpoint selection	Validation Macro-F1

The saved run uses a CPU. After all three epochs, the checkpoint with the highest validation Macro-F1 is loaded for final test evaluation.

5.3 Core implementation

Listing 5 converts articles into BERT inputs. The tokenizer returns `input_ids`, attention masks, and any other model inputs. The attention mask identifies padding positions. The dataset adds integer `labels`, allowing the sequence-classification model to compute cross-entropy internally. Separate datasets and loaders use the shared partitions; training batches are shuffled, while validation/test batches are unshuffled.

Listing 5. BERT tokenization and dataset .

```
bert_tokenizer = AutoTokenizer.from_pretrained(
    BERT_NAME, use_fast=True
)

class NewsDataset(Dataset):
    def __init__(self, texts, labels):
        self.encodings = bert_tokenizer(
            texts, truncation=True,
            padding="max_length",
            max_length=MAX_LENGTH,
            return_tensors="pt",
        )
        self.labels = torch.tensor(
            labels, dtype=torch.long
        )

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, index):
        item = {
            key: tensor[index]
            for key, tensor in self.encodings.items()
        }
        item["labels"] = self.labels[index]
        return item

bert_checkpoint = (
    ARTIFACT_DIR / "bert_best_state.pt"
)
```

Listing 6 initializes the pre-trained encoder with a three-class head. The label mappings preserve the business/politics/sports interpretation. Passing `bert_model.parameters()` to AdamW includes the encoder and the head; the notebook does not freeze either component.

Listing 6. BERT model and optimizer initialization .

```
bert_model = \
    AutoModelForSequenceClassification.from_pretrained(
        BERT_NAME,
        num_labels=len(class_names),
        id2label={
            i: label
            for i, label in enumerate(class_names)
        },
        label2id={
            label: i
            for i, label in enumerate(class_names)
        },
    ).to(device)
optimizer = torch.optim.AdamW(
    bert_model.parameters(),
    lr=2e-5, weight_decay=0.01,
)
```

The abridged loop in Listing 7 retains model updates and checkpoint selection. `train()` enables training behaviour, `zero_grad` clears gradients from the preceding batch, and `backward()` differentiates the classification loss. Gradient norm clipping precedes `optimizer.step()`. The original loop also accumulates loss and stores the history in Table 6; those logging operations and the progress bar are omitted here.

Listing 7. Fine-tuning and checkpoint selection (logging omitted).

```
best_val_f1 = -1.0
for epoch in range(1, BERT_EPOCHS + 1):
    bert_model.train()
    for batch in train_loader:
        batch = {
            key: value.to(device)
            for key, value in batch.items()
        }
        optimizer.zero_grad(set_to_none=True)
        output = bert_model(**batch)
        output.loss.backward()
        torch.nn.utils.clip_grad_norm_(
            bert_model.parameters(), max_norm=1.0
        )
        optimizer.step()
    val_pred = predict_bert(bert_model, val_loader)
    val_f1 = f1_score(
        y_val, val_pred,
        average="macro", zero_division=0,
    )
    if val_f1 > best_val_f1:
        best_val_f1 = val_f1
        torch.save(
            bert_model.state_dict(), bert_checkpoint
        )
```

`predict_bert`, defined in the BERT setup cell, runs under `@torch.no_grad()`, calls `eval()`, removes labels from model inputs, and chooses `logits.argmax(dim=-1)`. The next epoch restores training mode. A checkpoint is replaced only when validation Macro-F1 strictly improves, so test performance cannot affect checkpoint selection.

Listing 8 restores the saved weights before evaluating the test loader. The revised cell first checks that the checkpoint exists. If `bert_model` is absent, it constructs the matching architecture from `AutoConfig`, then loads all saved weights; this supports checkpoint evaluation without repeating fine-tuning in the current kernel. `map_location=device` loads tensors onto the selected device; the `TypeError` fallback supports older PyTorch versions that lack `weights_only`. The final evaluation uses the same metric helper as Tasks 1 and 2.

Listing 8. Best-checkpoint loading and test evaluation .

```
if not bert_checkpoint.is_file():
    raise FileNotFoundError(bert_checkpoint)

if "bert_model" not in globals():
    bert_config = AutoConfig.from_pretrained(
        BERT_NAME,
        num_labels=len(class_names),
        id2label={
            i: label
            for i, label in enumerate(class_names)
        },
        label2id={
            label: i
            for i, label in enumerate(class_names)
        },
    )
    bert_model = (
        AutoModelForSequenceClassification
        .from_config(bert_config).to(device)
    )

try:
    best_state = torch.load(
        bert_checkpoint, map_location=device,
        weights_only=True,
    )
except TypeError:
    best_state = torch.load(
        bert_checkpoint, map_location=device
    )
bert_model.load_state_dict(best_state)
del best_state
bert_pred = predict_bert(bert_model, test_loader)
evaluate_predictions(
    "BERT-base-uncased fine-tuned", y_test, bert_pred
)
```

5.4 Training dynamics and test performance

Training loss decreases from 0.1440 to 0.0215, while validation performance is not monotonic (Table 6). Validation Macro-F1 decreases in epoch 2 and recovers to its highest value in epoch 3. Thus, the third checkpoint is selected. It obtains **98.26% test Accuracy** and **96.05% test Macro-F1**, corresponding to 1,132 correct predictions out of 1,152 test articles.

Table 6. BERT training history; validation scores are percentages.

Epoch	Train loss	Val. Acc.	Val. Macro-F1
1	0.1440	97.66	94.83
2	0.0443	97.22	93.73
3	0.0215	97.92	**95.51**

5.5 Class-level error analysis

Figure 1 presents the saved confusion matrix. The color is normalized within each true class, while the annotations retain raw counts and row percentages. This prevents the much larger sports class from visually dominating the minority-class errors.

There are 20 errors: 12 business, four politics, and four sports articles. Ten business articles are predicted as politics, compared with three politics articles predicted as business. The largest off-diagonal cell is therefore the business-to-politics direction. The class F1 scores recomputed from these counts are 94.24%, 94.31%, and 99.60% for business, politics, and sports, respectively. These results show that confusion is concentrated between business and politics, whereas

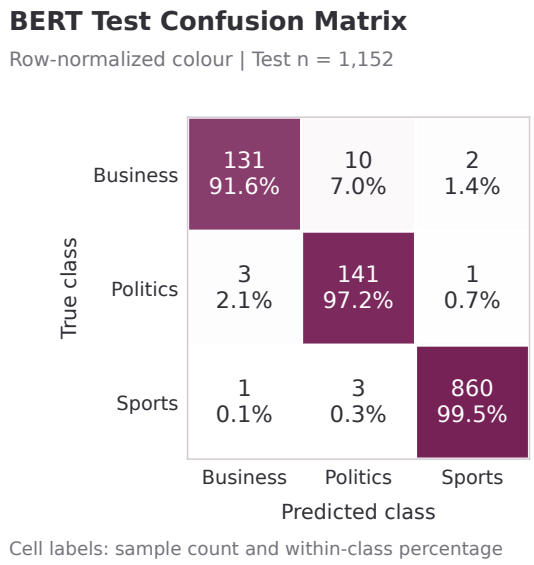


Figure 1. BERT confusion matrix on the NYT test set. Rows are true classes; columns are predictions. Percentages use each true-class row total.

sports is more clearly distinguished. Business and political news often discuss overlapping topics, such as economic policy, government regulation, and international trade, which may make their distinction more difficult. Sports articles tend to contain distinctive vocabulary related to teams, players, and competitions, providing clearer classification cues. This pattern is reflected in the class F1 scores: sports reaches 99.60%, substantially higher than business and politics.

6 Comparative Discussion

6.1 Overall performance

Figure 2 summarizes all six test results. The explicitly labelled 94–100% range makes small differences visible. Binary BoW with LR has the highest Macro-F1; NYT Word2Vec is the strongest mean-embedding pipeline and ties Binary BoW in Accuracy at the displayed precision. BERT's Macro-F1 is 1.23 percentage points below Binary BoW. Accuracy exceeds Macro-F1 for every method, consistent with stronger overall performance than balanced minority-class performance.

The results show that a simple lexical classifier can provide a strong baseline for news topic classification. BERT is fine-tuned for only three epochs with a maximum input length of 64 tokens, which may limit both task adaptation and access to article content. More expressive representations therefore do not automatically yield higher scores under these input and training constraints.

7 Conclusion

The three tasks implement a progression from sparse lexical features to static embeddings and contextual fine-tuning. Under the shared NYT split, Binary BoW with LR achieves the highest recorded Macro-F1 and ties NYT Word2Vec in Accuracy at the displayed precision. Training Word2Vec on NYT yields the strongest averaged-embedding result, while the required three-epoch BERT run remains effective despite its short input limit. The confusion matrix shows that BERT's

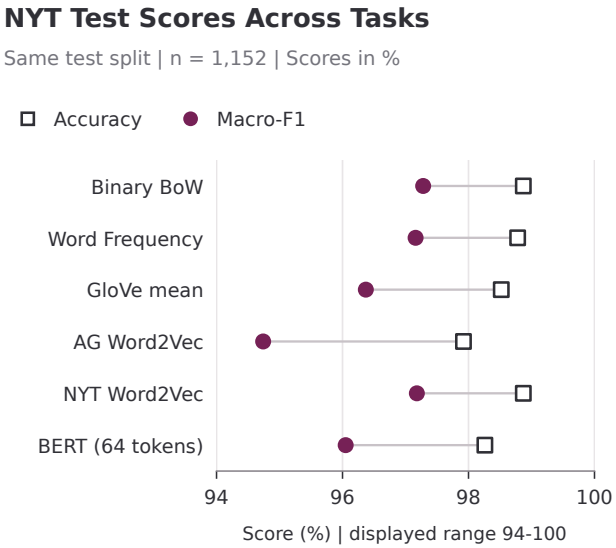


Figure 2. Accuracy and Macro-F1 for all six methods on the same NYT test split. Open squares denote Accuracy; filled circles denote Macro-F1.

remaining errors are concentrated more heavily in business and politics than in sports.

The main lesson from this assignment is to evaluate representation choices with a strong baseline and a clear protocol. Balanced metrics, vocabulary coverage, input constraints, and data independence all help interpret the scores.

References

[1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

[2] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[3] Jeffrey Pennington, Richard Socher, and Christopher D Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543, 2014.